

Eco Vision V2 — Estimativa de Cobertura Vegetal por Processamento Digital de Imagens

Computação Gráfica | Avaliação A3 | Processamento Digital de Imagens Ambientais com Python

⚠ Os campos marcados com [PREENCHER] dependem da imagem real e dos seus dados pessoais — complete-os antes de exportar para PDF.

1. Identificação do Projeto

Campo	Informação
Título do Projeto	Eco Vision V2 — Estimativa de Cobertura Vegetal por Processamento Digital de Imagens
Aluno	Keyllian Azevedo
Matrícula	[PREENCHER]
Turma	[PREENCHER]
Professor(a)	[PREENCHER]
Disciplina	Computação Gráfica
Curso	Ciência da Computação
Tipo de Avaliação	A3 — Projeto Prático Individual
Semestre	2025.1
Data de Entrega	[PREENCHER]
Linguagem	Python 3.10+
Bibliotecas Principais	OpenCV (headless), NumPy, Matplotlib, Flask
Plataforma	Aplicação web local (Flask) — IDE VSCode
Repositório	[PREENCHER — URL do GitHub]

2. Introdução e Contextualização

O monitoramento da cobertura vegetal é um dos problemas centrais da agenda ambiental contemporânea: a perda de vegetação está diretamente associada ao desmatamento, à degradação de ecossistemas e à redução da qualidade de vida nos centros urbanos. Quantificar **quanto de uma área ainda é vegetação** — e acompanhar a evolução desse número ao longo do tempo — é o primeiro passo para qualquer ação de preservação ou recuperação.

Este projeto, denominado **Eco Vision V2**, aplica técnicas de Computação Gráfica e Processamento Digital de Imagens para responder a essa pergunta de forma automática. A partir de uma única imagem ambiental, o sistema executa um pipeline completo — aquisição, pré-processamento, análise de histograma, filtragem, detecção de bordas e segmentação — e entrega como resultado final o **percentual estimado de cobertura vegetal da cena**.

O sistema foi implementado como uma **aplicação web interativa** (Flask), na qual cada etapa do pipeline é executada e exibida individualmente, com comparações lado a lado entre a imagem de entrada e a imagem processada. Essa arquitetura torna o processo didático e transparente: é possível observar e explicar o efeito de cada técnica antes de avançar para a próxima.

3. Objetivos

3.1 Objetivo Geral

Desenvolver um sistema de processamento digital de imagens em Python capaz de estimar o percentual de cobertura vegetal de uma cena ambiental, aplicando técnicas de Computação Gráfica para extrair informações com fins analíticos e ambientais.

3.2 Objetivos Específicos

- Capturar/obter uma imagem ambiental com participação ativa do aluno, documentando o processo e os equipamentos utilizados;
- Implementar rotinas de pré-processamento: leitura, conversão de espaço de cores (BGR → RGB → escala de cinza), redimensionamento e normalização;
- Aplicar e comparar técnicas distintas de processamento: filtros de suavização (Gaussiano, Mediana e Bilateral), detecção de bordas (Canny), transformações morfológicas e análise de histograma;
- Segmentar as regiões de vegetação combinando o índice ExG (adaptação do NDVI para imagens RGB) com a limiarização automática de Otsu;
- Apresentar resultados comparativos (original × processado) com análise crítica dos efeitos de cada técnica;
- Quantificar a cobertura vegetal da cena e calcular métricas de qualidade (PSNR e SNR).

4. Fundamentação Teórica

4.1 Imagem Digital

Uma imagem digital é uma representação matricial de uma cena, composta por pixels. Em imagens coloridas no modelo RGB, cada pixel é descrito por três canais — Vermelho (R), Verde (G) e Azul (B) — com valores entre 0 e 255 (8 bits por canal). No projeto, as imagens são manipuladas como matrizes NumPy de dimensões `altura × largura × 3`.

4.2 Espaços de Cores

- **RGB** — modelo aditivo padrão; usado para exibição e para o cálculo do índice de vegetação;
- **BGR** — ordem interna do OpenCV; convertida para RGB na aquisição;
- **Escala de Cinza** — intensidade única por pixel, obtida pela média ponderada $0,299R + 0,587G + 0,114B$, que reflete a sensibilidade do olho humano; é a entrada do detector de bordas.

4.3 Técnicas Aplicadas no Projeto

Técnica	Descrição	Aplicação no projeto
Filtro Gaussiano	Convolução com kernel gaussiano ($\sigma=1,5$); reduz ruído de alta frequência	Suavização comparativa
Filtro de Mediana	Substituição pelo valor mediano da vizinhança 5×5; preserva bordas	Remoção de ruído impulsivo
Filtro Bilateral	Pondera proximidade espacial E semelhança de cor; não borra bordas	Suavização com preservação de contornos
Detecção de Bordas (Canny)	Gradiente multi-direção, supressão de não-máximos e histerese (50/150)	Contorno de regiões
Índice ExG (NDVI adaptado)	$ExG = 2G - R - B$; realça pixels onde o verde domina	Mapa de vegetação para segmentação
Thresholding (Otsu)	Limiar automático que maximiza a variância entre classes	Segmentação binária vegetação × não-vegetação
Morfologia Matemática	Abertura e fechamento com elemento estruturante elíptico 5×5	Refino da máscara de vegetação
Análise de Histograma	Distribuição de intensidades por canal R, G e B	Caracterização da cena

Sobre o índice ExG: o NDVI clássico (Normalized Difference Vegetation Index) requer a banda do infravermelho próximo, indisponível em câmeras RGB comuns. O ExG (*Excess Green*) é a adaptação consagrada para esse cenário: mede, pixel a pixel, o excesso do canal verde sobre os demais — assinatura espectral típica da vegetação no espectro visível.

5. Metodologia

5.1 Origem e Captura da Imagem

Esta seção vale 15% da nota — deve evidenciar participação ativa do aluno.

Campo	Informação
Tipo de obtenção	[PREENCHER — ex.: fotografia própria com celular / download de satélite INPE-Sentinel]
Local retratado	[PREENCHER — ex.: Praça X, Cidade-UF / coordenadas]
Data e hora da captura	[PREENCHER]
Equipamento / plataforma	[PREENCHER — ex.: smartphone modelo Y, câmera 50 MP / Sentinel Hub]
Condições	[PREENCHER — ex.: dia ensolarado, foto tomada de ponto elevado]
Justificativa da escolha	[PREENCHER — por que esta cena é adequada para estimar cobertura vegetal]

[PREENCHER — descreva em um parágrafo o processo de captura/busca e seleção, anexando prints se for download de satélite]

5.2 Ambiente de Desenvolvimento

Componente	Especificação
Linguagem	Python 3.12
Ambiente de execução	Aplicação web Flask (navegador) — desenvolvida no VSCode (Linux)
Biblioteca principal	OpenCV-Python headless 4.x (cv2)
Visualização	Matplotlib 3.x
Manipulação numérica	NumPy 2.x
Servidor web	Flask 3.x

5.3 Pipeline de Processamento

O sistema executa 11 etapas interativas, avançadas uma a uma pelo usuário no painel web:

1. **Aquisição** — upload pelo navegador, decodificação, gravação de `imagem_original.png` e conversão BGR → RGB com exibição de metadados;
2. **Pré-processamento** — escala de cinza, redimensionamento para 800 px de largura (proporção preservada) e normalização [0, 1];
3. **Análise de histograma** — distribuição dos canais R, G e B, médias por canal e identificação do canal dominante;
4. **Filtro Gaussiano** ($\sigma = 1,5$);
5. **Filtro de Mediana** (kernel 5×5);
6. **Filtro Bilateral** ($d = 9$, $\sigma_{\text{cor}} = \sigma_{\text{espaco}} = 75$);
7. **Deteção de bordas** — Canny com limiares 50 e 150 e suavização prévia;
8. **Índice de vegetação ExG** — mapa $2G - R - B$ normalizado para [0, 255];
9. **Limiarização de Otsu** — sobre o mapa ExG, gerando a máscara binária;
10. **Refino morfológico** — abertura + fechamento com elipse 5×5;
11. **Resultado final** — contornos da vegetação sobre a original, percentual de cobertura vegetal e métricas PSNR/SNR.

6. Estrutura de Implementação

O código é organizado em um pacote `processamento/` com um módulo por responsabilidade, orquestrado pelo servidor Flask (`app.py`):

Módulo / Função	Finalidade	Biblioteca
<code>aquisicao.carregar_imagem()</code>	Lê e converte BGR → RGB	OpenCV
<code>pre_processamento.pre_processar()</code>	Escala de cinza, resize, normalização	OpenCV + NumPy
<code>histograma.analisar_histograma()</code>	Distribuição de intensidades por canal	OpenCV + Matplotlib
<code>filtros.aplicar_filtros()</code>	Gaussiano, Mediana e Bilateral	OpenCV
<code>bordas.detectar_bordas()</code>	Canny com suavização prévia	OpenCV

Módulo / Função	Finalidade	Biblioteca
segmentacao.indice_vegetacao()	Mapa ExG (2G – R – B)	NumPy + OpenCV
segmentacao.segmentar()	Otsu + morfologia + contornos	OpenCV
metricas.calcular_metricas()	PSNR, SNR e cobertura vegetal	NumPy
visualizacao.plotar_grid()	Grids comparativos do relatório	Matplotlib

Instalação e execução:

```
pip install -r requirements.txt
python app.py # abre o painel em http://localhost:5000
```

7. Resultados Obtidos

Insira aqui as figuras geradas em `resultados/` após processar a imagem real.

7.1 Imagem Original e Metadados

[PREENCHER — inserir imagem_original.png]

Metadado	Valor
Resolução	[PREENCHER] px
Canais	3 (RGB)
Tamanho	[PREENCHER] KB

7.2 Histogramas dos Canais R, G e B

[PREENCHER — inserir resultados/01_histograma_rgb.png]

Médias por canal: R = [PREENCHER] , G = [PREENCHER] , B = [PREENCHER] . Canal dominante: [PREENCHER] .

[PREENCHER — análise da distribuição: a cena é clara ou escura? O domínio de algum canal é coerente com o conteúdo (verde → vegetação)?]

7.3 Original × Escala de Cinza

[PREENCHER — inserir resultados/00_original_vs_cinza.png]

7.4 Comparativo dos Filtros de Suavização

[PREENCHER — inserir resultados/02_filtros_comparativo.png]

7.5 Detecção de Bordas (Canny)

[PREENCHER — inserir resultados/03_bordas_canny.png]

Percentual de pixels de borda: [PREENCHER] %.

7.6 Segmentação — Máscara antes e após Morfologia

[PREENCHER — inserir resultados/04_segmentacao_otsu.png]

Limiar de Otsu encontrado (sobre o ExG): [PREENCHER] . Regiões de vegetação detectadas: [PREENCHER] contornos.

7.7 Contornos Sobrepostos e Resumo do Pipeline

[PREENCHER — inserir resultados/05_resumo_pipeline.png]

7.8 Tabela de Métricas

Métrica	Valor
---------	-------

Métrica	Valor
Cobertura vegetal estimada	[PREENCHER] %
PSNR (original × filtrada)	[PREENCHER] dB
SNR (original × filtrada)	[PREENCHER] dB
Valor médio de pixel (original)	[PREENCHER]

8. Análise Crítica dos Resultados

Roteiro de discussão — substitua as orientações pelas observações feitas na SUA imagem.

Filtros de suavização. [PREENCHER — compare os três filtros na sua imagem: o Gaussiano borrou bordas visivelmente? A Mediana removeu pontos isolados? O Bilateral preservou os contornos das copas/folhas? Qual seria o mais adequado como pré-processamento para a segmentação e por quê?]

Deteção de bordas. [PREENCHER — os limiares 50/150 capturaram os contornos relevantes? Há bordas espúrias causadas por textura (grama, folhas)? O que aconteceria ao reduzir/aumentar os limiares?]

Segmentação da vegetação. [PREENCHER — a máscara final corresponde à vegetação visível? Houve falsos positivos (objetos verdes que não são plantas) ou falsos negativos (vegetação seca/amarelada não detectada)? Qual foi o efeito visível da morfologia na limpeza da máscara?]

Resultado ambiental. [PREENCHER — o percentual de cobertura vegetal estimado é plausível para a cena? Como esse número poderia ser usado em um monitoramento real (comparação temporal da mesma área)?]

Limitações. O índice ExG depende da assinatura verde da vegetação: vegetação seca, sombras intensas e objetos artificiais verdes podem afetar a estimativa. [PREENCHER — limitações observadas no seu caso]

9. Conclusão

[PREENCHER — parágrafo final retomando: o problema (medir cobertura vegetal), a solução (pipeline ExG + Otsu + morfologia via painel web), o resultado obtido (X% na cena analisada) e possíveis trabalhos futuros (comparação temporal, NDVI com banda infravermelha, segmentação por clustering K-means, suporte multi-usuário no deploy)]

10. Referências Bibliográficas

- [1] GONZALEZ, R. C.; WOODS, R. E. **Processamento Digital de Imagens**. 3. ed. São Paulo: Pearson, 2010.
- [2] BRADSKI, G.; KAEHLER, A. **Learning OpenCV 4: Computer Vision with Python 3**. O'Reilly Media, 2019.
- [3] SZELISKI, R. **Computer Vision: Algorithms and Applications**. 2. ed. Springer, 2022.
- [4] WOEBBECKE, D. M. et al. **Color indices for weed identification under various soil, residue, and lighting conditions**. Transactions of the ASAE, v. 38, n. 1, p. 259–269, 1995. (artigo original do índice ExG)
- [5] OTSU, N. **A threshold selection method from gray-level histograms**. IEEE Transactions on Systems, Man, and Cybernetics, v. 9, n. 1, p. 62–66, 1979.
- [6] OpenCV Documentation. Disponível em: <https://docs.opencv.org/>. Acesso em 2025.
- [7] NumPy Documentation. Disponível em: <https://numpy.org/doc/>. Acesso em 2025.
- [8] Flask Documentation. Disponível em: <https://flask.palletsprojects.com/>. Acesso em 2025.
- [9] INPE — Instituto Nacional de Pesquisas Espaciais. Catálogo de Imagens. Disponível em: <http://www.inpe.br/>. Acesso em 2025.